

Neural Networks Deep Learning

Semester Assignments 2026

Theodora Tzina
tzinatheod@ece.auth.gr
10715

Professor:
Anastasios Tefas



Contents

01. **Datasets in use**

02. **K – Nearest Neighbors
& Nearest Class Centroid**

03. **Convolutional Neural
Network & Multi Layer
Perceptron**

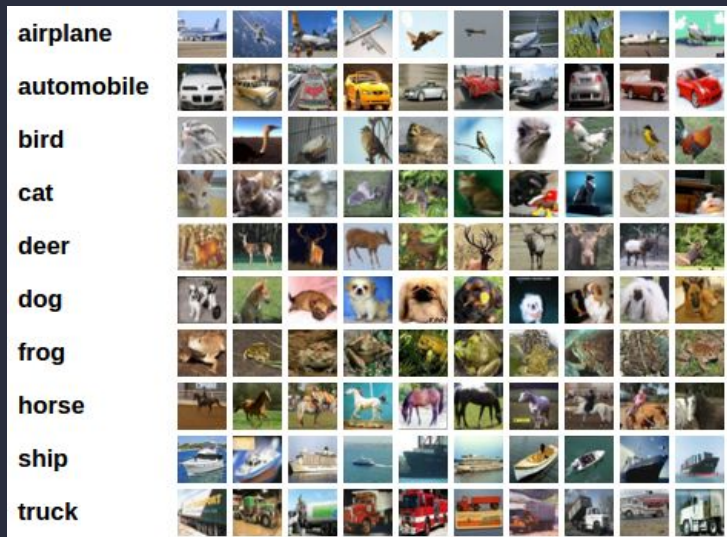
04. **Support Vector
Machines**

05. **Autoencoders**

06. **Comparison results**

Datasets in use

CIFAR - 10

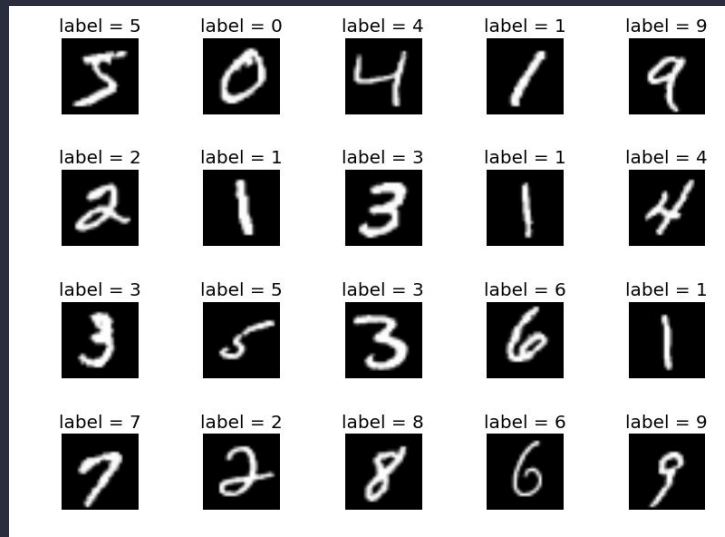


Dataset Size: 50,000 Training Set + 10,000 Test Set

Perfectly Balanced: 6,000 images per class

Image Dimensions: 32x32 pixels + 3 color channels (RGB)

MNIST

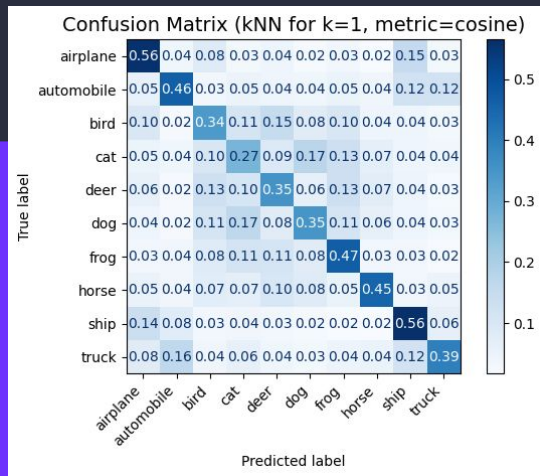


Dataset Size: 60,000 Training Set + 10,000 Test Set

Relatively Balanced: ~7,000 images per class

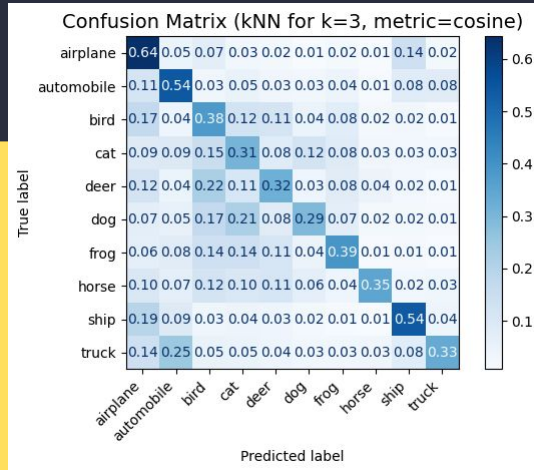
Image Dimensions: 28x28 pixels + 1 color channel (grayscale)

K - Nearest Neighbors



kNN for k=1, Cosine distance

Test accuracy: 42.11%



kNN for k=3, Cosine distance

Test accuracy: 40.97%

Distance Metrics tested:

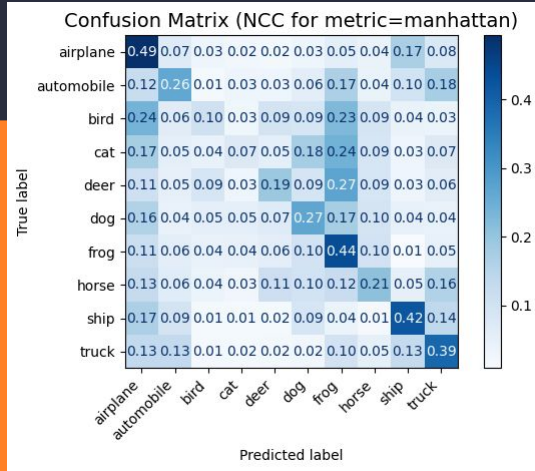
- Euclidean
- Cosine
- Manhattan

Best case models for
Cosine distance



PCA reduced dimensions to 50
for both kNN, NCC - Explained
variance = **84.3%**

Nearest Class Centroid



Distance Metrics tested:

- Euclidean
- Manhattan

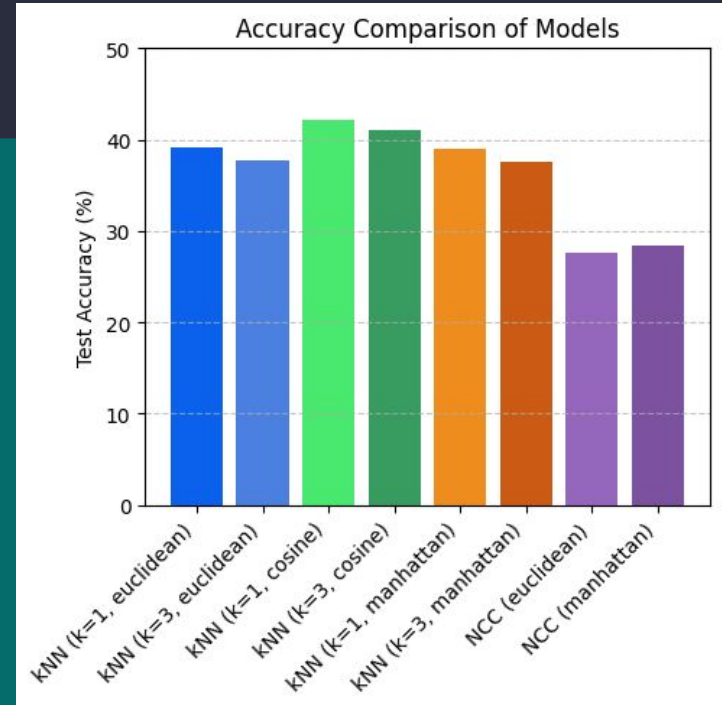
Best case model for **Manhattan** distance



NCC, Manhattan distance

Test accuracy: 28.47%

→ Classification reports were also printed for both kNN, NCC



Accuracy Comparison kNN + NCC

CNN

Best case model:

Accuracy **89.72%** – **57** epochs – Training Time **6521.26 s**

Layers: Conv2D (32, 64, 128, 256 filters), MaxPooling2D, BatchNormalization, GlobalAveragePooling2D, Dropout (0.3 only for output), Dense (for output layers)

Activation functions: Mostly "ReLu", "Softmax" for output layer

Optimizer: Adam

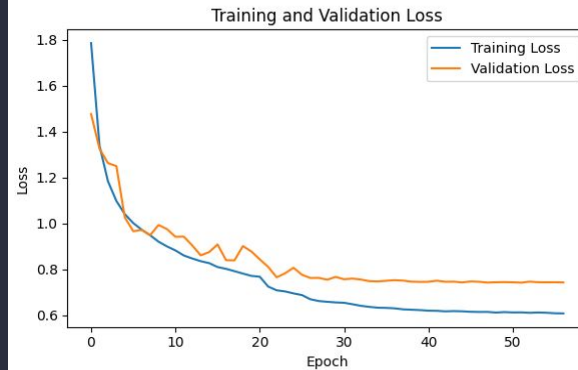
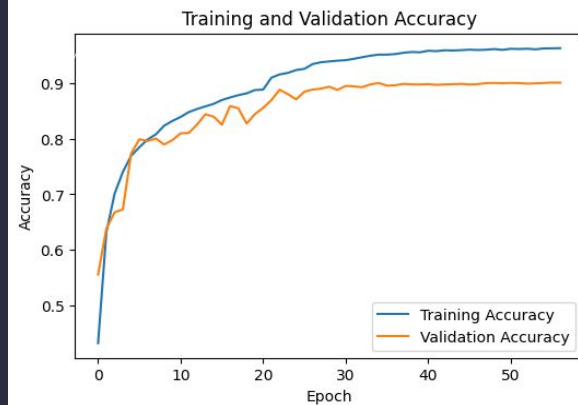
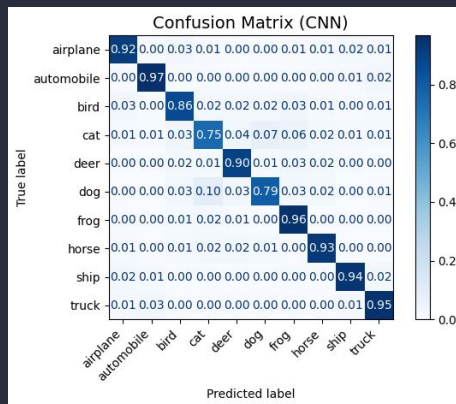
Loss: Sparse Categorical Crossentropy with Label Smoothing (custom)

Additional techniques: Data Augmentation, Early Stopping, Learning Rate Scheduler

TensorFlow/Keras was used to build the model, which was optimized by:

- **Hyperparameter Tuning**
- **Trial-Error method**

Validation split was set to **0.2** for the training set, in order to keep test set untouched for the final evaluation of each model.



MLP

Random base model:

Accuracy **49.49%** – **50** epochs – Training Time **640.59 s**

Layers: Dense (512, 256, 128, 10 for output), Flatten, BatchNormalization, Dropout (0.4)

Activation functions: Mostly "ReLu", "Softmax" for output layer

Optimizer: Adam

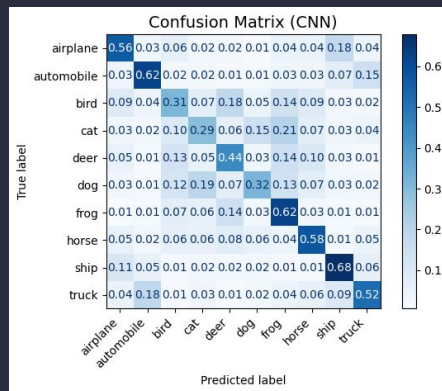
Loss: Sparse Categorical Crossentropy

Additional techniques: Data Augmentation, Early Stopping, Learning Rate Scheduler

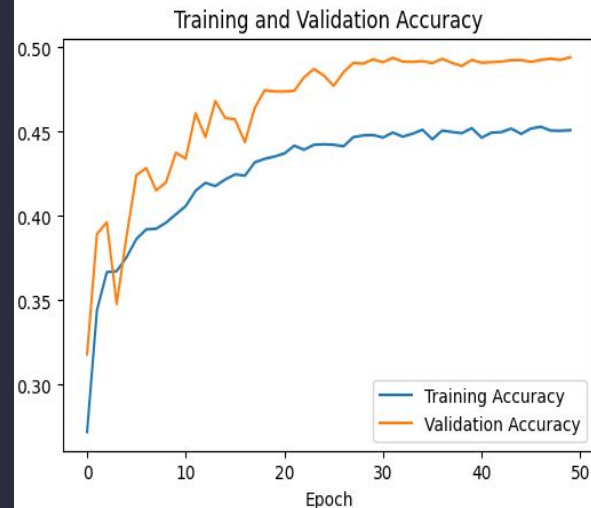
TensorFlow/Keras was used to build the model, based on:

→ Best CNN hyperparameters and techniques

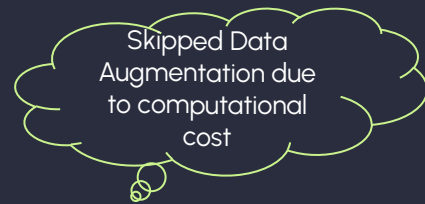
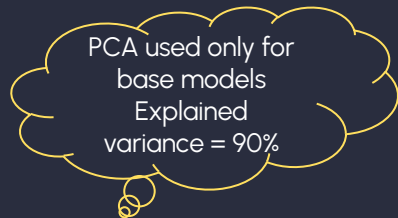
Validation split was set to **0.2** for the training set, in order to keep test set untouched for the final evaluation of each model.



We expect better performance on image datasets from CNNs, so a base MLP model was built only for comparison reasons:



SVM



2 Classes - Base

CIFAR-10 merged and balanced into 2 classes: Animals + Vehicles

Kernels: Linear, Polynomial, RBF, Sigmoid

Best model: RBF Kernel

- **Hyperparameters:** $C = 100$, $\gamma = 0.001$
- **Accuracy** = 88.37%

10 classes - Base

CIFAR-10 maintained with it's original classes

Kernels: Linear, Polynomial, RBF, Sigmoid

Best model: RBF Kernel

- **Hyperparameters:** $C = 1$, $\gamma = 0.01$
- **Accuracy** = 52.73%

2 Classes - Optimized

CIFAR-10 best 2-class SVM models optimized by additional techniques

Best model: RBF Kernel

- **Hyperparameters:** $C = 1$, $\gamma = \text{'scale'}$
- Data Augmentation
- Histogram of Oriented Gradients (HOG)
- Color Histogram Features
- **Accuracy** = 92.59%
- Support Vectors Ratio = 31.33%

kNN - NCC - MLP

Used best SVM model techniques for comparison

Accuracy:

- **kNN** ($k=1$): 84.74%
- **kNN** ($k=3$): 87.38%
- **NCC**: 81.76%
- **MLP** (1 hidden layer, Hinge loss): 90.2%

Validation split was set to **0.4** for the training set, in order to keep test set untouched for the final evaluation.

Autoencoder

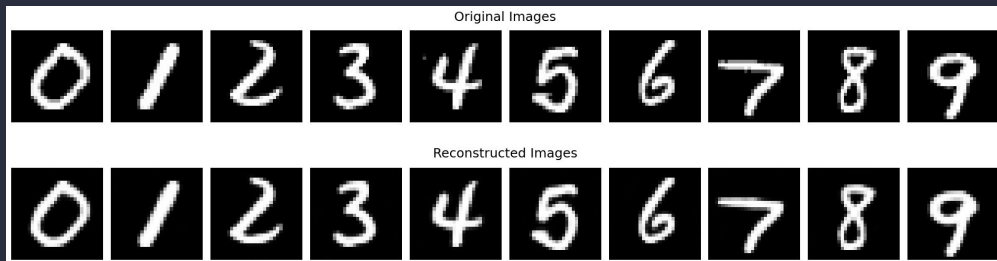
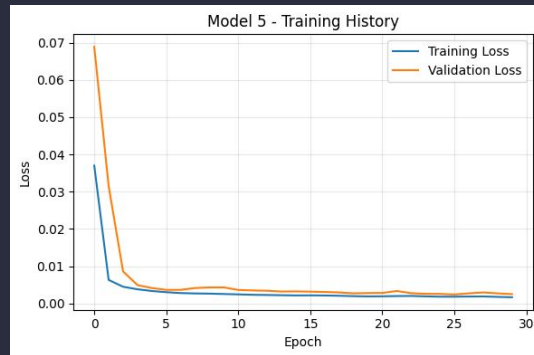
1. Digit Reconstruction

- **Model 1:** Dense layers (encoding dimension = 32)
- **Model 2:** Dense layers (encoding dimension = 64)
- **Model 3:** Dense layers (encoding dimension = 128)
- **Model 4:** Dense layers (encoding dimension = 128) + **PSNR, SSIM**
- **Model 5:** Convolutional model (encoding dimension = 512)

Best model:
Convolutional

→ **Test MSE:**
0.002452

→ **Time:**
2705.37
seconds



Validation split was set to 0.4 (for all autoencoders) for the training set, in order to keep test set untouched for the final evaluation of each model.

Autoencoder

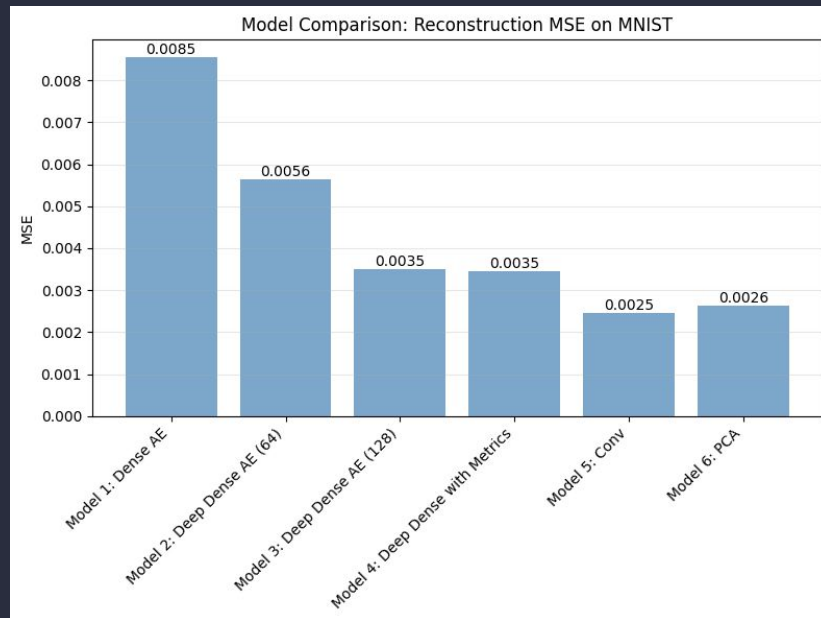
PCA Reconstruction for comparison:
784 $\rightarrow$ 154 $\rightarrow$ 784 (95% explained variance)

- $\rightarrow$ PCA Test MSE: 0.002632
- $\rightarrow$ Time: 4.86 seconds

CNN Classifier for reconstructed images:

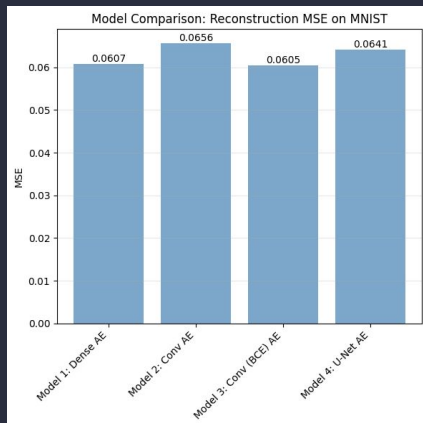
- $\rightarrow$ Accuracy on Original Images: 0.9910
- $\rightarrow$ Accuracy on Reconstructed Images: 0.9895

1. Digit Reconstruction



Autoencoder

- **Model 1:** Dense layers (encoding dimension = 128)
- **Model 2:** Convolutional model (encoding dimension = 512)
- **Model 3:** Convolutional model (encoding dimension = 512) + Binary Crossentropy Loss (**BCE**)
- **Model 4:** U-Net model with **Skip Connections** + Learning Rate Scheduler



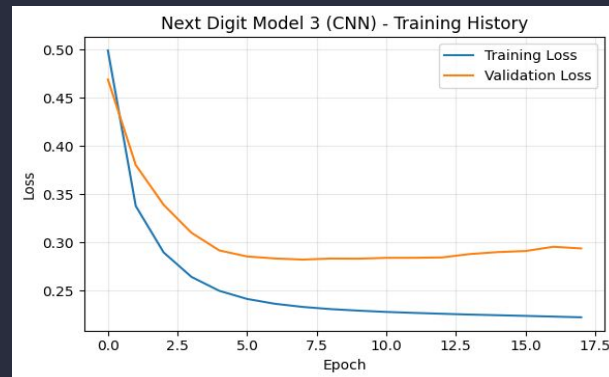
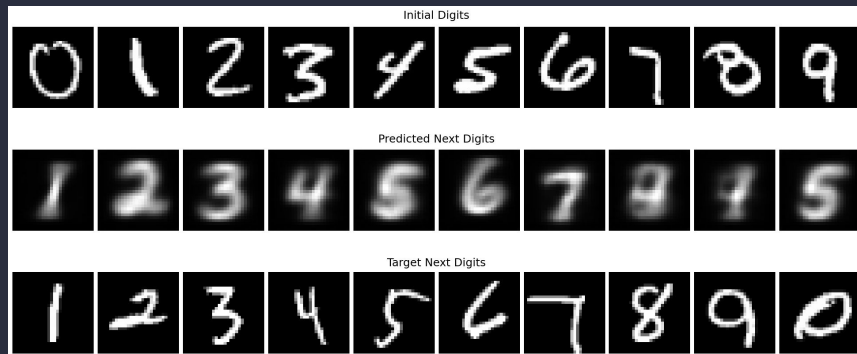
Best model: Convolutional + BCE

→ **Test MSE:** 0.060738

→ **Time:** 1925.83 seconds

Model still needs improvement,
last digits are wrongly
reconstructed

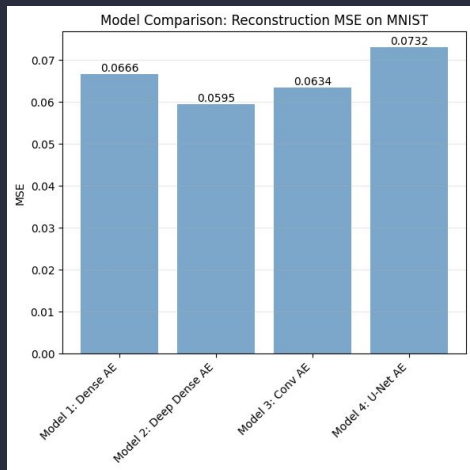
2. Next Digit Reconstruction



Autoencoder

3. Digit Addition Reconstruction

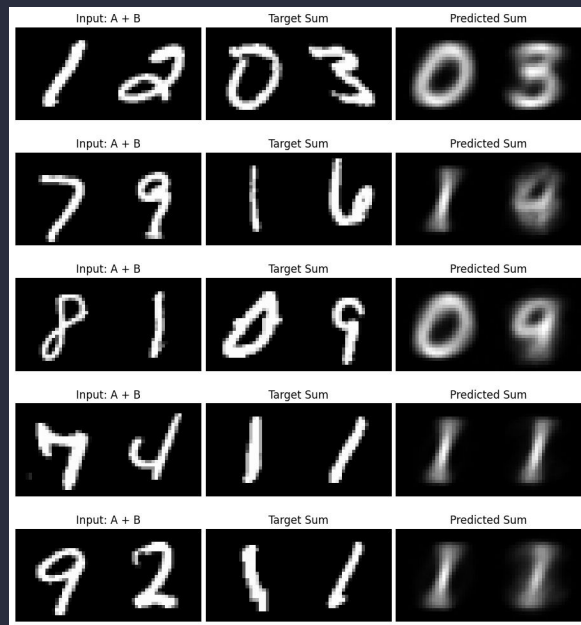
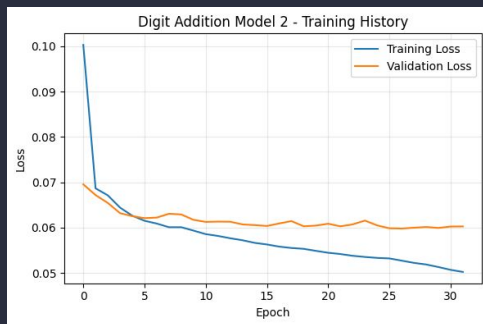
- **Model 1:** Dense layers (encoding dimension = 256)
- **Model 2:** Dense layers (encoding dimension = 512)
- **Model 3:** Convolutional model
- **Model 4:** U-Net model with **Skip Connections** + Learning Rate Scheduler



Best model: Dense (512)

→ **Test MSE:** 0.059476

→ **Time:** 177.88 seconds



Skip Connections failed due to:
Encoder input $\neq$ Decoder output

Comparison results

- **CNN vs. MLP:** CNN tend to achieve much higher accuracy compared to MLP because **spatial layers** (Conv2D) are superior to MLP's **flattened** approach for image data, though training times are significantly different.
- **SVM:** The SVM performance dropped significantly for **2 classes** compared to **10 classes**, proving that SVMs work best for a few classes where clear margins can be established.
- **Digit Reconstruction:** While **Convolutional Autoencoder** provided very accurate reconstruction, **PCA** reconstruction was nearly as effective and vastly more efficient.
- **Next Digit & Addition:** These complex tasks yielded higher error rates and proved that not all advanced architectures e.x. **Skip Connections** are suitable for tasks like these (Encoder input $\neq$ Decoder output).

Thank you

Theodora Tzina 10715